DSA4213 Natural Language Processing for Data Science

Doudou Zhou (ddzhou@nus.edu.sg)

Lecture 3 Neural Language Models and Recurrent Architectures

Lecture Plan

Lecture 3: Neural Language Models and Recurrent Architectures

NUS
National University
of Singapore

From Embeddings to Recurrent Language Models

Section 1

NLP

Statistics & Data Science

Embeddings alone do not define a language model

Lecture 1: predict the next token

$$P_\theta(w_{t+1} \mid w_{1:t})$$

An n-gram language model estimates this probability from exact suffix counts. Its weakness is sparsity: different but similar contexts do not share evidence.

Lecture 2: represent similar words by similar vectors

$$w_j \longrightarrow \mathbf{x}_j = \mathbf{E}^\top \mathbf{e}_{w_j} \in \mathbb{R}^d$$

Embeddings let related words, such as *students* and *pupils*, share statistical strength.

What is still missing

$$(w_1, \ldots, w_t) \longrightarrow (\mathbf{x}_1, \ldots, \mathbf{x}_t) \qquad \text{but not yet} \qquad P_\theta(w_{t+1} \mid w_{1:t}).$$

How can we use these vectors for next-token prediction?

Why keep only the last k tokens?

Language prefixes can have different lengths.

A prefix may contain two tokens, twenty tokens, or hundreds of tokens.

A feed-forward neural network expects a fixed-width input.

The simplest compromise is to keep only the last k tokens.

$$\underbrace{\text{~~as the proctor started the clock,~~}}_{\text{discarded history}}\ \underbrace{\textit{the students opened their}}_{k=4 \text{ input tokens}}\ \_\_\_\_$$

Every example now supplies exactly k input positions, so a feed-forward neural network can be used.

A fixed-window model maps k embeddings to one next-token distribution

Next-token distribution

$$\widehat{\mathbf{y}}_t = \text{softmax}(\mathbf{U}\mathbf{h}_t + \mathbf{b}_o)$$

Hidden representation

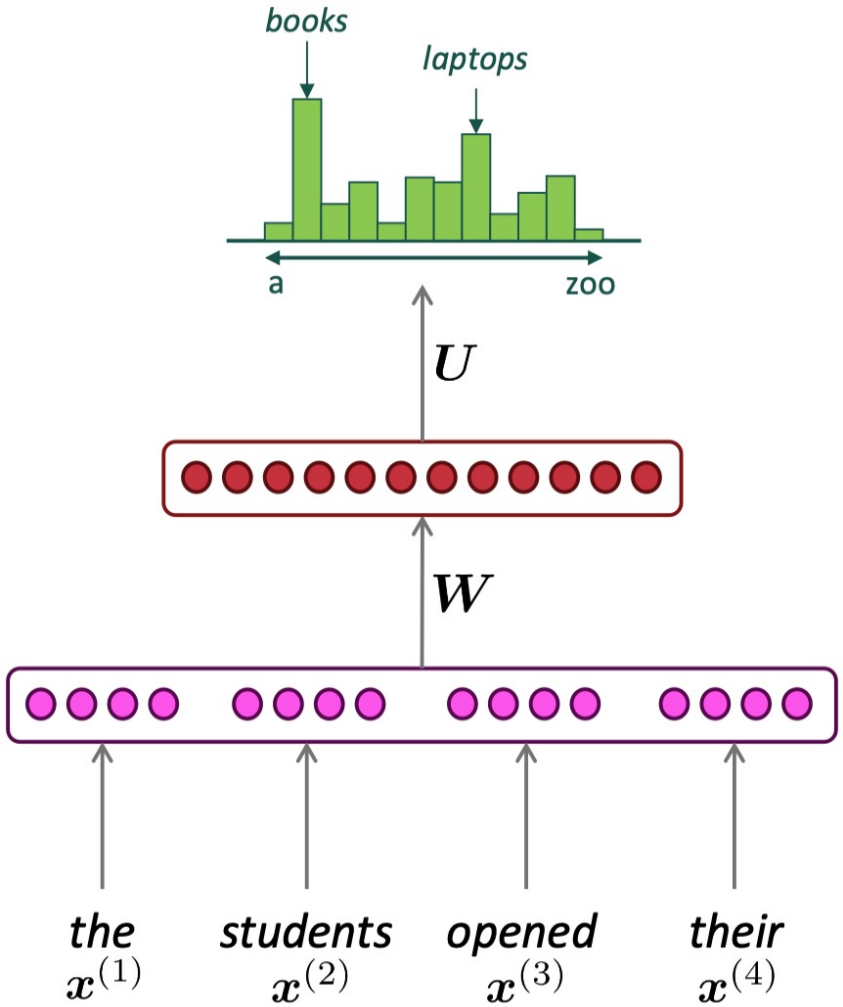
$$\mathbf{h}_t = f(\mathbf{W}\mathbf{e}_t + \mathbf{b}_h)$$

Concatenated embeddings

$$\mathbf{e}_t = [\mathbf{x}_{t-k+1}; \ldots; \mathbf{x}_t] \in \mathbb{R}^{kd}$$

Embedding lookup

$$\mathbf{x}_j = \mathbf{E}^\top \mathbf{e}_{w_j} \in \mathbb{R}^d$$



Why did word2vec become the landmark for word embeddings?

Before 2003: distributed representations already existed

Representing symbols by continuous vectors was an established neural-network idea. Word vectors were not invented by word2vec.

2003: Bengio et al. learned two components jointly

$$\underbrace{w_j \longmapsto \mathbf{x}_j}_{\text{word representation}} \quad + \quad \underbrace{(\mathbf{x}_{t-k+1}, \ldots, \mathbf{x}_t) \longmapsto P_\theta(w_{t+1} \mid w_{t-k+1:t})}_{\text{language model}}$$

The embeddings were parameters inside the language model, learned from the same next-token objective.

2013: word2vec made word-vector learning practical at scale

CBOW and skip-gram were simple enough to train on massive corpora, while the learned vectors still captured strong semantic and syntactic regularities.

Speed, scale, and clearly visible structure made word2vec the landmark work.

Sources: Bengio et al. (2003); Mikolov et al. (2013).

Fixed-window neural LM: what improves, what remains?

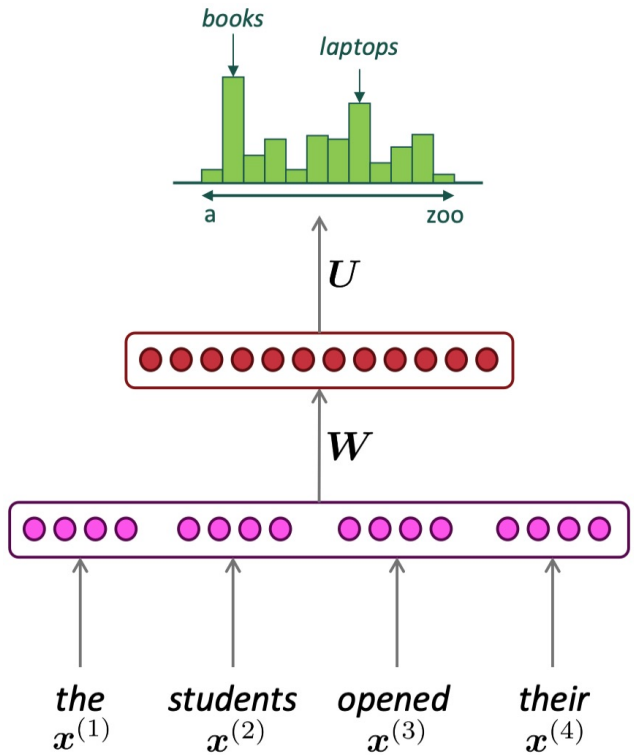
Based on Bengio et al. (2003): A Neural Probabilistic Language Model

Improvements over *n*-gram LM:

- Less sparsity: embeddings share statistical strength
- No need to store all observed *n*-grams

Remaining problems:

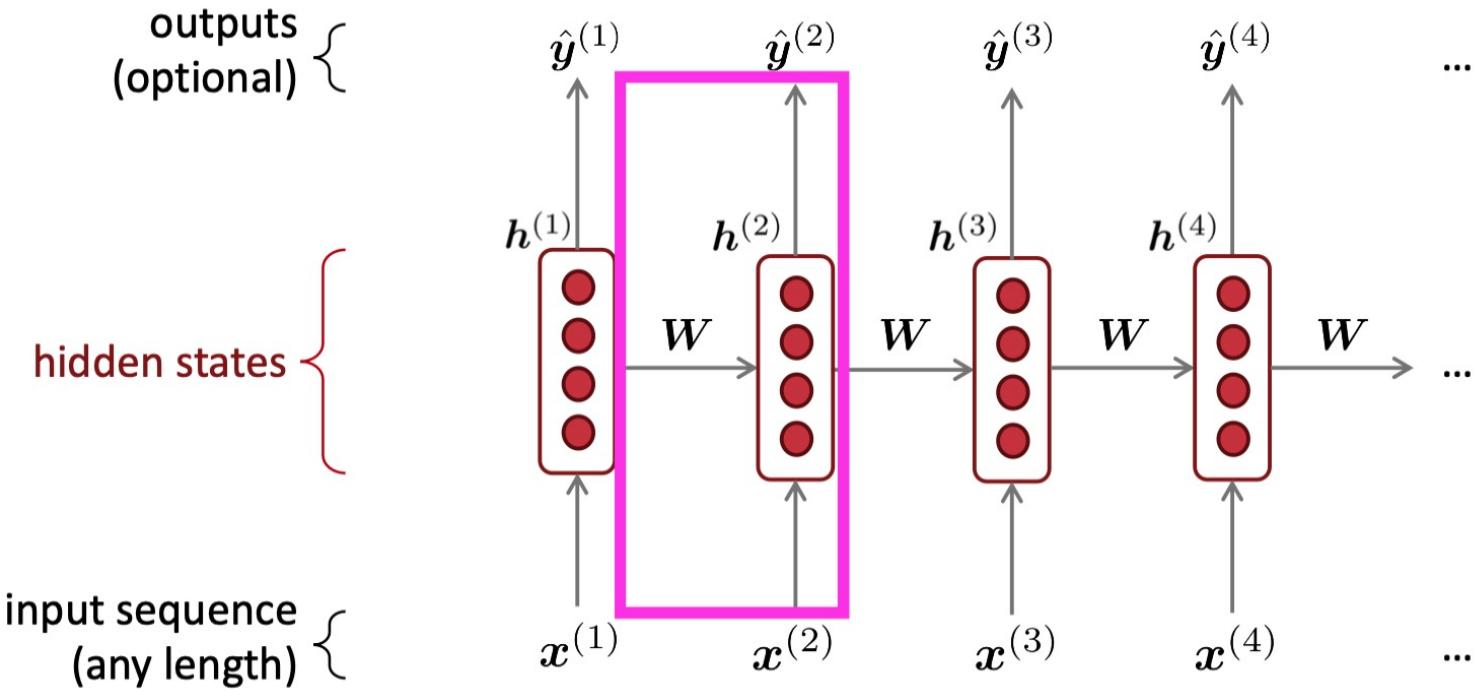
- A fixed window can miss relevant earlier context
- Larger window $\Rightarrow$ larger parameter matrix **W**
- Each window position is processed by different weights



We need one update rule for sequences of any length.

Recurrent Neural Networks (RNN)

- A family of neural architectures for sequences
- Core idea: apply the same weights **W** repeatedly over time



The figure writes $\mathbf{h}^{(t)}$; in our notation this is the state $\mathbf{h}_t$ at time t.

One recurrent update is reused at every position

- Embed the current token:

$$\mathbf{x}_t = \mathbf{E}^\top \mathbf{e}_{w_t}.$$

- Update the sequence state:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h).$$

- Predict the next token:

$$\widehat{\mathbf{y}}_t = \mathrm{softmax}(\mathbf{W}_o \mathbf{h}_t + \mathbf{b}_o).$$

Stage	Representation	Parameter shapes
Input	$\mathbf{e}_{w_t} \in \mathbb{R}^{\lvert V\rvert}$, $\mathbf{x}_t \in \mathbb{R}^d$	$\mathbf{E} \in \mathbb{R}^{\lvert V\rvert \times d}$
State	$\mathbf{h}_t \in \mathbb{R}^m$	$\mathbf{W}_x \in \mathbb{R}^{m \times d}$, $\mathbf{W}_h \in \mathbb{R}^{m \times m}$, $\mathbf{b}_h \in \mathbb{R}^m$
Output	$\widehat{\mathbf{y}}_t \in \mathbb{R}^{\lvert V\rvert}$	$\mathbf{W}_o \in \mathbb{R}^{\lvert V\rvert \times m}$, $\mathbf{b}_o \in \mathbb{R}^{\lvert V\rvert}$

The same parameter set is reused at every time step.

In-class check: trace one next-token prediction

For $w_{1:4} = \textit{the students opened their}$ and target $w_5 = \textit{laptops}$, recall that $\widehat{\mathbf{y}}_t$ predicts w_{t+1}.

1. Which hidden state is used to predict *laptops*?

In-class check: trace one next-token prediction

For $w_{1:4} =$ *the students opened their* and target $w_5 =$ *laptops*, recall that $\widehat{\mathbf{y}}_t$ predicts w_{t+1}.

1. Which hidden state is used to predict *laptops*?

   $\Rightarrow$ $\mathbf{h}_4$ produces $\widehat{\mathbf{y}}_4$, which predicts w_5.

In-class check: trace one next-token prediction

For $w_{1:4} =$ *the students opened their* and target $w_5 =$ *laptops*, recall that $\widehat{\mathbf{y}}_t$ predicts w_{t+1}.

1. Which hidden state is used to predict *laptops*?

   $\Rightarrow$ $\mathbf{h}_4$ produces $\widehat{\mathbf{y}}_4$, which predicts w_5.

2. Through which states can *students* affect that prediction?

In-class check: trace one next-token prediction

For $w_{1:4} =$ *the students opened their* and target $w_5 =$ *laptops*, recall that $\widehat{\mathbf{y}}_t$ predicts w_{t+1}.

1. Which hidden state is used to predict *laptops*?

   $\Rightarrow$ $\mathbf{h}_4$ produces $\widehat{\mathbf{y}}_4$, which predicts w_5.

2. Through which states can *students* affect that prediction?

   $\Rightarrow$ $\mathbf{x}_2 \longrightarrow \mathbf{h}_2 \longrightarrow \mathbf{h}_3 \longrightarrow \mathbf{h}_4 \longrightarrow \widehat{\mathbf{y}}_4$.

In-class check: trace one next-token prediction

For $w_{1:4} =$ *the students opened their* and target $w_5 =$ *laptops*, recall that $\widehat{\mathbf{y}}_t$ predicts w_{t+1}.

1. Which hidden state is used to predict *laptops*?

   $\Rightarrow$ $\mathbf{h}_4$ produces $\widehat{\mathbf{y}}_4$, which predicts w_5.

2. Through which states can *students* affect that prediction?

   $\Rightarrow$ $\mathbf{x}_2 \longrightarrow \mathbf{h}_2 \longrightarrow \mathbf{h}_3 \longrightarrow \mathbf{h}_4 \longrightarrow \widehat{\mathbf{y}}_4$.

3. Which parameters are reused across positions?

In-class check: trace one next-token prediction

For $w_{1:4} =$ *the students opened their* and target $w_5 =$ *laptops*, recall that $\widehat{\mathbf{y}}_t$ predicts w_{t+1}.

1. Which hidden state is used to predict *laptops*?

   $\Rightarrow$ $\mathbf{h}_4$ produces $\widehat{\mathbf{y}}_4$, which predicts w_5.

2. Through which states can *students* affect that prediction?

   $\Rightarrow$ $\mathbf{x}_2 \longrightarrow \mathbf{h}_2 \longrightarrow \mathbf{h}_3 \longrightarrow \mathbf{h}_4 \longrightarrow \widehat{\mathbf{y}}_4$.

3. Which parameters are reused across positions?

   $\Rightarrow$ $\mathbf{E}, \mathbf{W}_x, \mathbf{W}_h, \mathbf{W}_o, \mathbf{b}_h, \mathbf{b}_o$ are reused at every position.

In-class check: trace one next-token prediction

For $w_{1:4}$ = *the students opened their* and target w_5 = *laptops*, recall that $\widehat{\mathbf{y}}_t$ predicts w_{t+1}.

1. Which hidden state is used to predict *laptops*?

   $\Rightarrow$ $\mathbf{h}_4$ produces $\widehat{\mathbf{y}}_4$, which predicts w_5.

2. Through which states can *students* affect that prediction?

   $\Rightarrow$ $\mathbf{x}_2 \longrightarrow \mathbf{h}_2 \longrightarrow \mathbf{h}_3 \longrightarrow \mathbf{h}_4 \longrightarrow \widehat{\mathbf{y}}_4$.

3. Which parameters are reused across positions?

   $\Rightarrow$ $\mathbf{E}, \mathbf{W}_x, \mathbf{W}_h, \mathbf{W}_o, \mathbf{b}_h, \mathbf{b}_o$ are reused at every position.

4. After *laptops* is appended, what changes and what remains fixed?

In-class check: trace one next-token prediction

For $w_{1:4}$ = *the students opened their* and target w_5 = *laptops*, recall that $\widehat{\mathbf{y}}_t$ predicts w_{t+1}.

1. Which hidden state is used to predict *laptops*?

   $\Rightarrow$ $\mathbf{h}_4$ produces $\widehat{\mathbf{y}}_4$, which predicts w_5.

2. Through which states can *students* affect that prediction?

   $\Rightarrow$ $\mathbf{x}_2 \longrightarrow \mathbf{h}_2 \longrightarrow \mathbf{h}_3 \longrightarrow \mathbf{h}_4 \longrightarrow \widehat{\mathbf{y}}_4$.

3. Which parameters are reused across positions?

   $\Rightarrow$ $\mathbf{E}, \mathbf{W}_x, \mathbf{W}_h, \mathbf{W}_o, \mathbf{b}_h, \mathbf{b}_o$ are reused at every position.

4. After *laptops* is appended, what changes and what remains fixed?

   $\Rightarrow$ Compute $\mathbf{x}_5, \mathbf{h}_5, \widehat{\mathbf{y}}_5$; the same parameter set and dimensions are retained.

RNN language models

RNN Advantages:

- Can process **any length** input
- Can (in theory) use information from **many steps back**
- Model size **doesn't increase** with longer context
- Same weights each step $\Rightarrow$ **symmetry**

RNN language models

RNN Advantages:

- Can process **any length** input
- Can (in theory) use information from **many steps back**
- Model size **doesn't increase** with longer context
- Same weights each step $\Rightarrow$ **symmetry**

RNN Disadvantages:

- Sequential computation is **slow** (hard to parallelize)
- In practice, hard to learn very long dependencies

We will explain why.

NUS
National University
of Singapore

Training, Generation, and Evaluation

Section 2

NLP

Statistics & Data Science

Training an RNN language model: objective

For a tokenized training sequence $x_1, \ldots, x_T$, the state at position t predicts

$$\widehat{\mathbf{y}}_t(w) = P_\theta(w \mid x_{1:t}), \qquad w \in V.$$

The observed next token supplies the target:

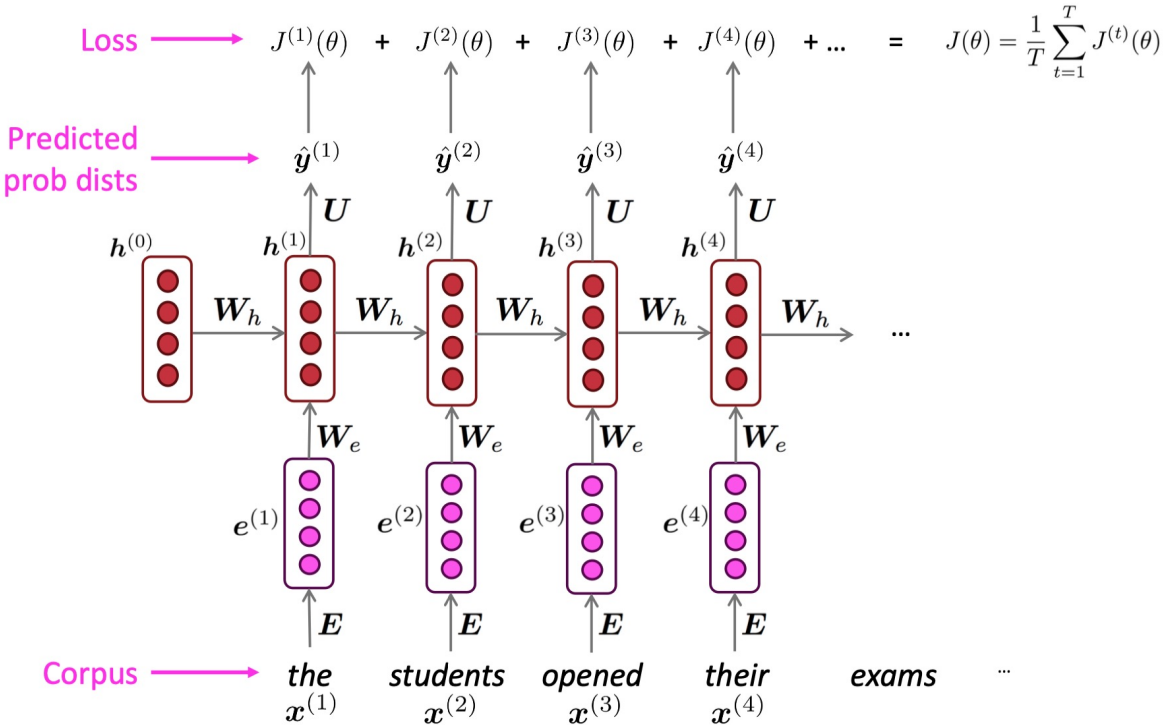
$$J^{(t)}(\theta) = -\log \widehat{\mathbf{y}}_t(x_{t+1}) = -\log P_\theta(x_{t+1} \mid x_{1:t}).$$

Average over the $T-1$ available next-token predictions:

$$J(\theta) = \frac{1}{T-1} \sum_{t=1}^{T-1} J^{(t)}(\theta).$$

- V: vocabulary; θ: all shared model parameters
- The same objective is used whether the recurrent update is a vanilla RNN or an LSTM.

Every prefix supplies one next-token training example



$$J(\theta) = \frac{1}{T-1} \sum_{t=1}^{T-1} \underbrace{-\log P_\theta(x_{t+1} \mid x_{1:t})}_{\text{loss from prefix } x_{1:t}}.$$

One sequence produces many supervised targets, while all positions share the same model parameters.

Sequence chunks and mini-batches solve different problems

Training uses the whole corpus over many parameter updates.

Each update processes only a manageable mini-batch of shorter sequences or chunks:

1. **Choose a manageable sequence length L**
   - A short sentence can be one training sequence
   - A long document is split into chunks of L tokens
2. **Put B sequences or chunks into one mini-batch**
3. **Compute their loss, update the parameters, and continue**

L = tokens in each sequence; B = sequences processed together.

Training uses the observed prefix at every step

Training

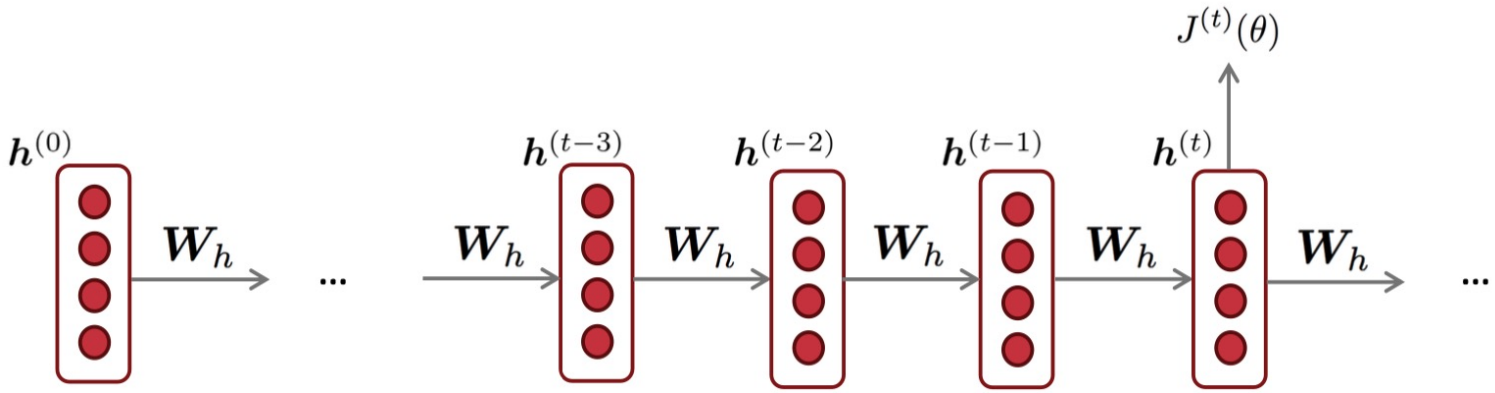
- Input at step t: true token x_t
- Target: true next token x_{t+1}
- Loss is available at every position

Autoregressive generation

- Input at the next step: the token produced by the model
- An early error changes all later contexts
- Decoding becomes part of observed behavior

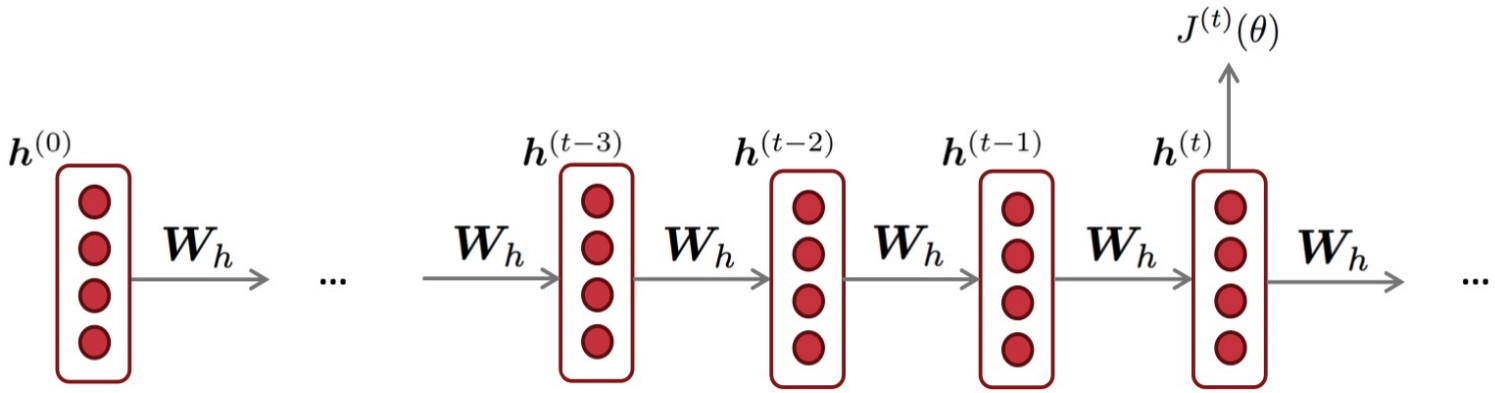
Using the observed previous token during training is commonly called **teacher forcing**.

Backpropagation for RNNs



Key question: What is the gradient of $J^{(t)}(\theta)$ with respect to the **shared** weight matrix $\mathbf{W}_h$?

Backpropagation for RNNs



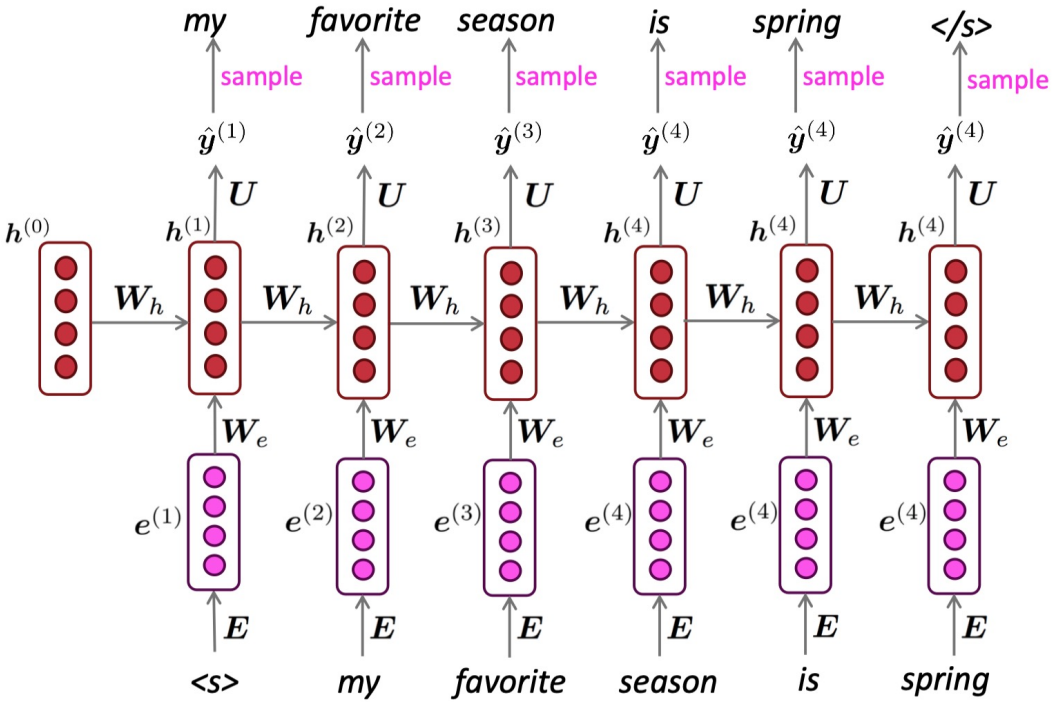
Key question: What is the gradient of $J^{(t)}(\theta)$ with respect to the **shared** weight matrix $\mathbf{W}_h$?

$$\frac{\partial J^{(t)}}{\partial \mathbf{W}_h} = \sum_{i=1}^{t} \left. \frac{\partial J^{(t)}}{\partial \mathbf{W}_h} \right|_{(i)}$$

When a parameter is reused multiple times, its gradient is the sum of its contributions from every place it is used.

Generating with an RNN Language Model

- After training, we can **generate text** using the language model
- Key idea:
  - At each step, **sample** a word from the predicted distribution
  - Feed the sampled word back as the next input
- This is different from training:
  - During training: next input is the **true word**
  - During generation: next input is the **model's own output**



Temperature controls the randomness of RNN generation

Greedy decoding can repeat, while sampling directly from the model distribution can be too random. Temperature controls this trade-off without retraining the model.

At step t, the RNN produces the next-token distribution

$$p_t(w) = P_\theta(w \mid x_{1:t}).$$

For temperature $T > 0$, define the distribution used for sampling as

$$q_{t,T}(w) = \frac{p_t(w)^{1/T}}{\sum_{u \in \mathcal{V}} p_t(u)^{1/T}}, \qquad w \in \mathcal{V},$$

where $\mathcal{V}$ is the vocabulary.

T	*laptops*	*books*	*exams*	Effect
0.5	0.73	0.22	0.05	sharper
1	0.55	0.30	0.15	unchanged
2	0.44	0.33	0.23	flatter

A two-step toy example shows why generation is autoregressive

Prefix	$P(\textit{laptops})$	$P(\textit{books})$	$P(\textit{exams})$
the students opened their	0.55	0.30	0.15

Suppose sampling chooses *books*. The model then updates its state with that chosen token:

$$\mathbf{h}_{t+1} = f(\mathbf{h}_t, \mathbf{x}_{books}),$$

and produces a new distribution, for example

$$P(\textit{and} \mid \textit{the students opened their books}) = 0.42.$$

One selected token changes the state and therefore every later distribution.

An RNN can imitate political style before it preserves meaning

Political-speech example

- Train a character-level RNN on a collection of Obama speeches.
- Generate a continuation one character at a time.

"The United States will step up to the cost of a new challenges of the American people that will share the fact. . ."

The cadence sounds plausible; the meaning quickly becomes unclear.

Source: Obama-RNN — Machine generated political speeches.

Familiar characters do not guarantee a coherent story

Fiction example

- Train a recurrent language model on the first four Harry Potter books.
- Ask it to generate a new chapter.

"Sorry," Harry shouted, panicking —
"I'll leave those brooms in London, are they?"
"No idea," said Nearly Headless Nick...

The names and dialogue feel familiar, but no stable event sequence emerges.

Source: Harry Potter: Written by Artificial Intelligence.

A real RNN can learn format without global semantic consistency

Structured-text example: generated recipe

- Train an RNN-LM on a dataset of recipes
- Ask it to generate a new recipe

Title: CHOCOLATE RANCH BARBECUE
Categories: Game, Casseroles, Cookies
Yield: 6 Servings
2 tb Parmesan cheese – chopped
1 c Coconut milk
3 Eggs, beaten

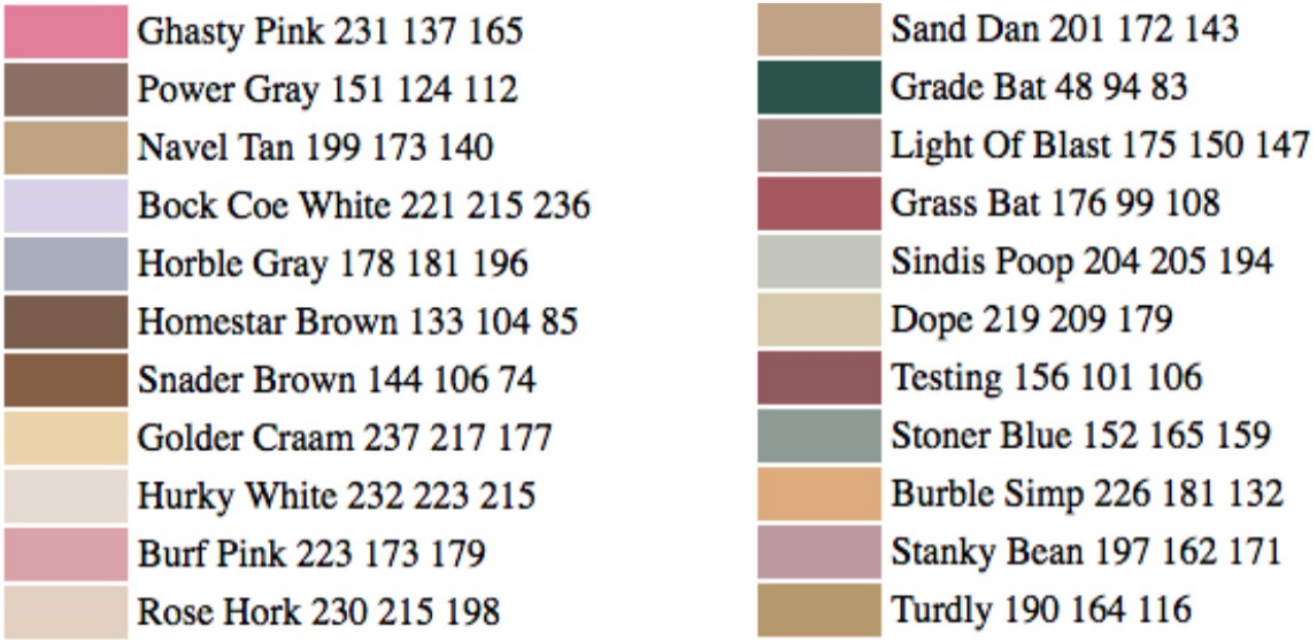
. . .

Source: gist.github.com/nylki

Character-level RNNs learn spelling patterns

Generated paint-color names

- Train a **character-level** RNN-LM
- Task: predict the next character
- Dataset: paint color names



Character-level RNNs model spelling and morphology directly

Source: aiweirdness.com

What did we learn from these examples?

- RNN language models can:
  - Learn style, tone, and local patterns
  - Produce fluent short sequences
- But they struggle with:
  - Long-range coherence
  - Global planning and consistency
- But one generated example is not a model evaluation:
  - The output depends on the prompt, sampling, and temperature
  - It does not summarize prediction over a test set

How can we evaluate next-token prediction over an entire test set?

Perplexity is the exponentiated average test loss

For N predicted tokens in a tokenized test sequence,

$$J_{\text{test}} = \frac{1}{N} \sum_{t=1}^{N} -\log P_\theta(x_{t+1} \mid x_{1:t}), \qquad \text{PPL} = \exp(J_{\text{test}}).$$

- $x_{1:t}$: the observed tokenized prefix
- x_{t+1}: the true next token
- N: the number of next-token predictions included in evaluation

If $J_{\text{test}} = \log 20$, then PPL $= 20$: the loss is comparable to facing about 20 equally plausible choices at each position.

Lower perplexity means better next-token prediction on the same evaluation setup.

Perplexity comparisons require the same prediction problem

Lower perplexity means lower average next-token loss only when the comparison holds fixed:

- the evaluation corpus and preprocessing;
- the vocabulary and tokenization;
- which tokens contribute to the loss; and
- whether external context or adaptation is allowed.

Perplexity is a property of a model–data–tokenization combination, not an architecture alone.

RNNs greatly improved perplexity over what came before

n-gram model →

Increasingly complex RNNs ↓

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

↓ Perplexity improves (lower is better)

Source:

https://research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words/

NUS
National University
of Singapore

What Vanilla RNNs Can Represent—and Why They Struggle

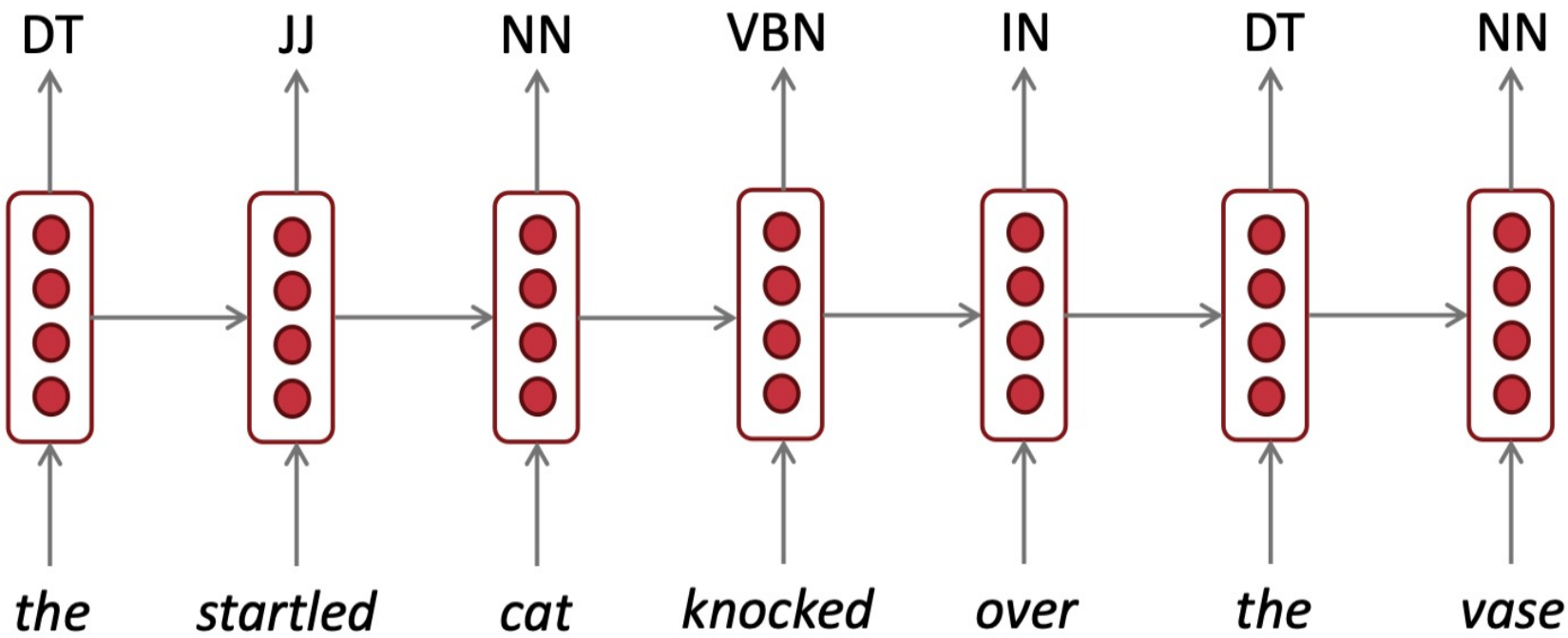
Section 3

NLP

Statistics & Data Science

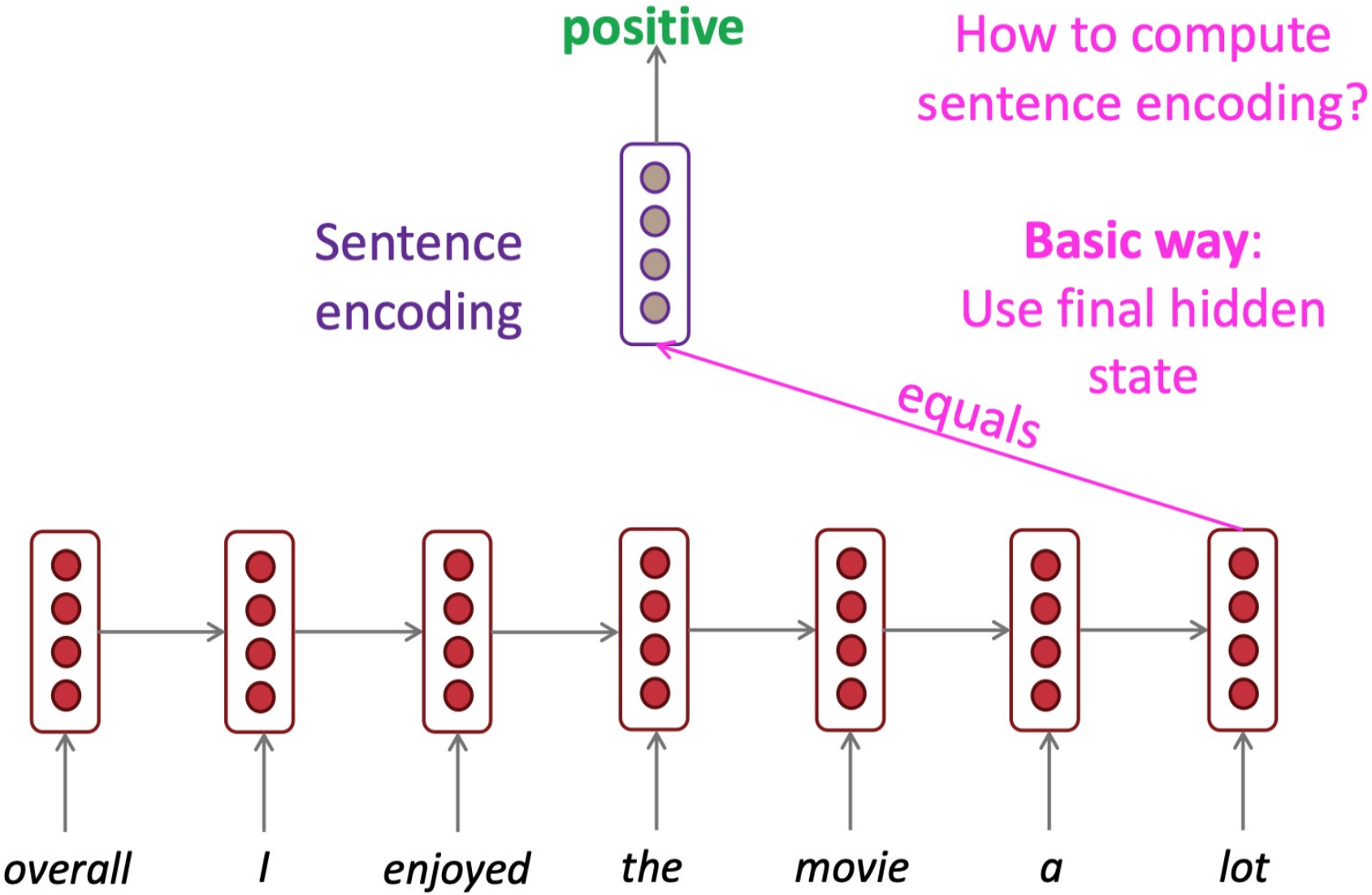
RNNs can be used for sequence tagging

e.g., part-of-speech tagging, named entity recognition



The final state can summarize a sequence for classification

e.g., sentiment classification

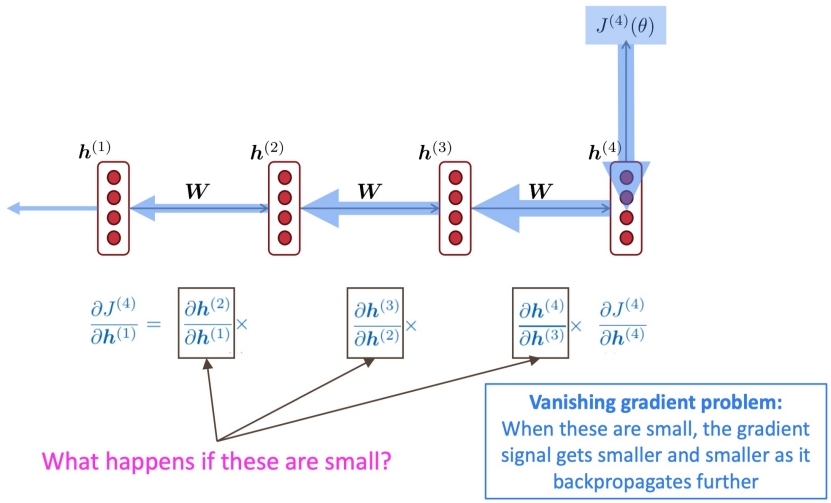


Can the final loss teach the RNN to preserve early information?

In the classification example, early information must travel forward to the final state. During learning, the gradient travels in the opposite direction:

$$\frac{\partial J^{(t)}}{\partial \mathbf{h}_i} = \frac{\partial J^{(t)}}{\partial \mathbf{h}_t} \prod_{j=i+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}.$$

- $t - i$: number of recurrent steps in the gradient path
- Each step contributes one Jacobian factor.



The same Jacobian product can vanish or explode

Consider a one-dimensional illustration in which every step multiplies the gradient by the same factor a:

$$\left|\frac{\partial J^{(t)}}{\partial h_{t-m}}\right| \approx |a|^m \left|\frac{\partial J^{(t)}}{\partial h_t}\right|.$$

Per-step factor	$m = 10$ steps	$m = 50$ steps
$a = 0.8$	$0.8^{10} \approx 0.107$	$0.8^{50} \approx 1.4 \times 10^{-5}$
$a = 1.2$	$1.2^{10} \approx 6.19$	$1.2^{50} \approx 9.1 \times 10^{3}$

- Contracting products cause **vanishing gradients**: early states receive little learning signal.
- Expanding products cause **exploding gradients**: parameter updates can become unstable.

Exploding gradients make parameter updates unstable

SGD updates the parameters by

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \underbrace{\nabla_\theta J(\theta)}_{\text{gradient}}.$$

- If $\|\nabla_\theta J(\theta)\|$ is very large, one update can jump to a region with much higher loss.
- Continued growth can produce **Inf** or **NaN** values and make the training run unusable.

The direction may be useful; the step is simply far too large.

Gradient clipping controls exploding gradients

Let $\mathbf{g} = \nabla_\theta J(\theta)$ and choose a threshold $\tau > 0$. Use

$$\widetilde{\mathbf{g}} = \min\left(1, \frac{\tau}{\|\mathbf{g}\|}\right)\mathbf{g}$$

in the parameter update.

- If $\|\mathbf{g}\| \leq \tau$, the gradient is unchanged.
- Otherwise, its norm is reduced to τ without changing direction.

Algorithm 1 Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|}\hat{\mathbf{g}}$
end if

Clipping controls explosion, but it cannot enlarge a gradient that has already vanished.

Clipping cannot recover a vanished learning signal

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her ____.*
- The final loss should teach the model to preserve the earlier word *tickets* across many recurrent steps.
- If the gradient reaching that early position is nearly zero, training provides almost no pressure to preserve it.

The dependency is representable in principle, but difficult to learn.

A separate memory path can preserve information more directly

Vanilla RNNs rewrite one hidden state at every step:

$$\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h).$$

- Forward information passes through another nonlinear transformation at every token.
- The backward learning signal passes through the corresponding chain of Jacobians.
- We therefore want a path that can carry selected information with only limited modification.

The LSTM adds a gated memory path with learned forget, write, and expose decisions.

Gated Memory and Gradient Flow

Section 4

NLP

Statistics & Data Science

LSTM adds a gated memory path to the recurrent state

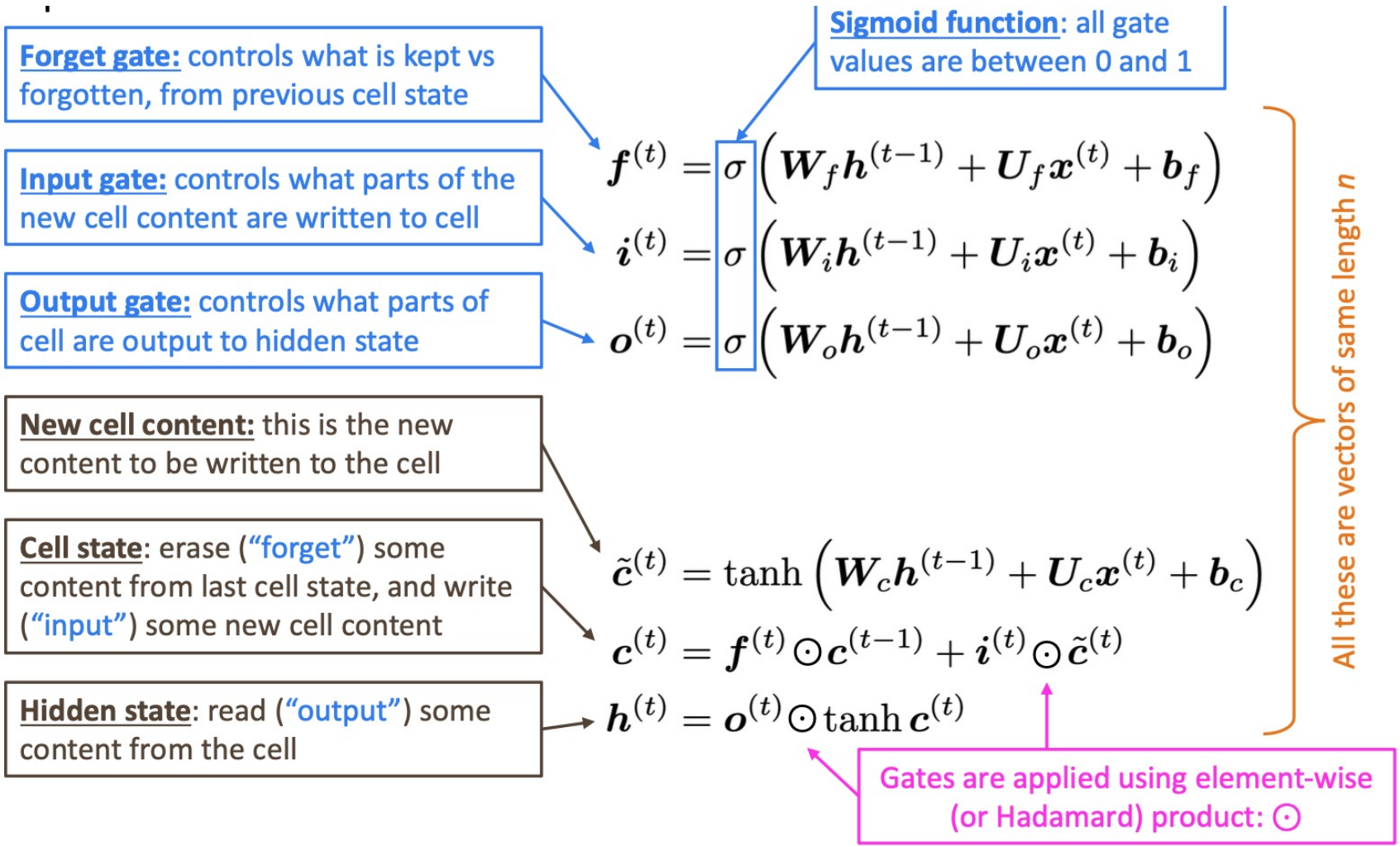
Vanilla RNNs rewrite one hidden state through a nonlinear transformation at every step. LSTM separates two roles:

- **Cell state** $\mathbf{c}_t \in \mathbb{R}^m$: a memory path that can be carried forward with limited modification
- **Hidden state** $\mathbf{h}_t \in \mathbb{R}^m$: the information exposed to prediction and the next recurrent update
- **Learned gates in** $[0,1]^m$**:** coordinate-wise decisions to preserve, write, or expose information

$$(\mathbf{h}_{t-1}, \mathbf{c}_{t-1}, \mathbf{x}_t) \longmapsto (\mathbf{h}_t, \mathbf{c}_t).$$

Introduced by Hochreiter and Schmidhuber (1997); the forget gate was added by Gers et al. (2000).

One LSTM step decides what to forget, write, and expose



Read the diagram from the three gates to the updated cell and hidden states.

How can an LSTM keep a value unchanged?

Focus on one coordinate of the cell state. Its update is

$$c_t = f_t c_{t-1} + i_t \widetilde{c}_t.$$

Suppose

$$c_{t-1} = 0.8$$

and we want the new cell value c_t to remain close to 0.8.

What should the forget gate f_t and input gate i_t do?

How can an LSTM keep a value unchanged?

Focus on one coordinate of the cell state. Its update is

$$c_t = f_t c_{t-1} + i_t \widetilde{c}_t.$$

Suppose

$$c_{t-1} = 0.8$$

and we want the new cell value c_t to remain close to 0.8.

What should the forget gate f_t and input gate i_t do?

Answer

$$f_t \approx 1, \qquad i_t \approx 0,$$

because

$$c_t \approx 1 \times 0.8 + 0 \times \widetilde{c}_t = 0.8.$$

The forget gate preserves the old value, while the input gate prevents new content from overwriting it.

Vanilla RNN and LSTM make different memory commitments

Vanilla RNN

- One hidden state
- Complete nonlinear update each step
- Fewer parameters
- Long memory must emerge from repeated use of $\mathbf{W}_h$

LSTM

- Hidden state plus cell state
- Explicit preserve, write, and read controls
- More parameters and computation
- A direct cell path supports longer information flow

LSTMs make long memory easier to learn; they do not guarantee that every useful dependency will be retained.

Long gradient products affect all deep networks

- Gradient instability is **not unique to RNNs**. It can occur in any deep network (feed-forward, CNNs, or RNNs), especially when the computation is deep.
- Let $h^{(\ell)}$ be the representation after layer ℓ, for $\ell = 0, \ldots, L$, and let $\mathcal{L}$ be the loss.
- **Why it happens:** backprop involves **products of Jacobians**

$$\frac{\partial \mathcal{L}}{\partial h^{(0)}} = \left(\prod_{\ell=1}^{L} \frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}} \right) \frac{\partial \mathcal{L}}{\partial h^{(L)}}$$

  - If typical factors have norm $< 1 \Rightarrow$ **vanishing gradients**
  - If typical factors have norm $> 1 \Rightarrow$ **exploding gradients**
  - Result: early layers learn slowly or training becomes unstable

- **Common fix:** create **shorter gradient paths** through skip or residual connections.

Residual connections provide an identity path through depth

Example: Residual connections (ResNet)

- Add an identity shortcut: $h^{(\ell+1)} = h^{(\ell)} + F(h^{(\ell)})$
- **Identity path** lets information/gradients pass through
- Makes **very deep networks** much easier to optimize

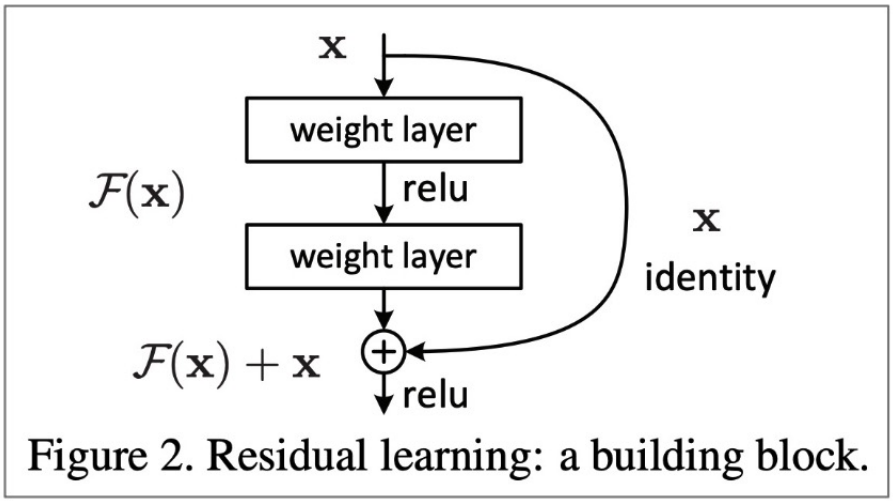


Figure 2. Residual learning: a building block.

He et al., 2015. "Deep Residual Learning for Image Recognition." Link

NUS
National University
of Singapore

Recurrent Variants and Remaining Limitations

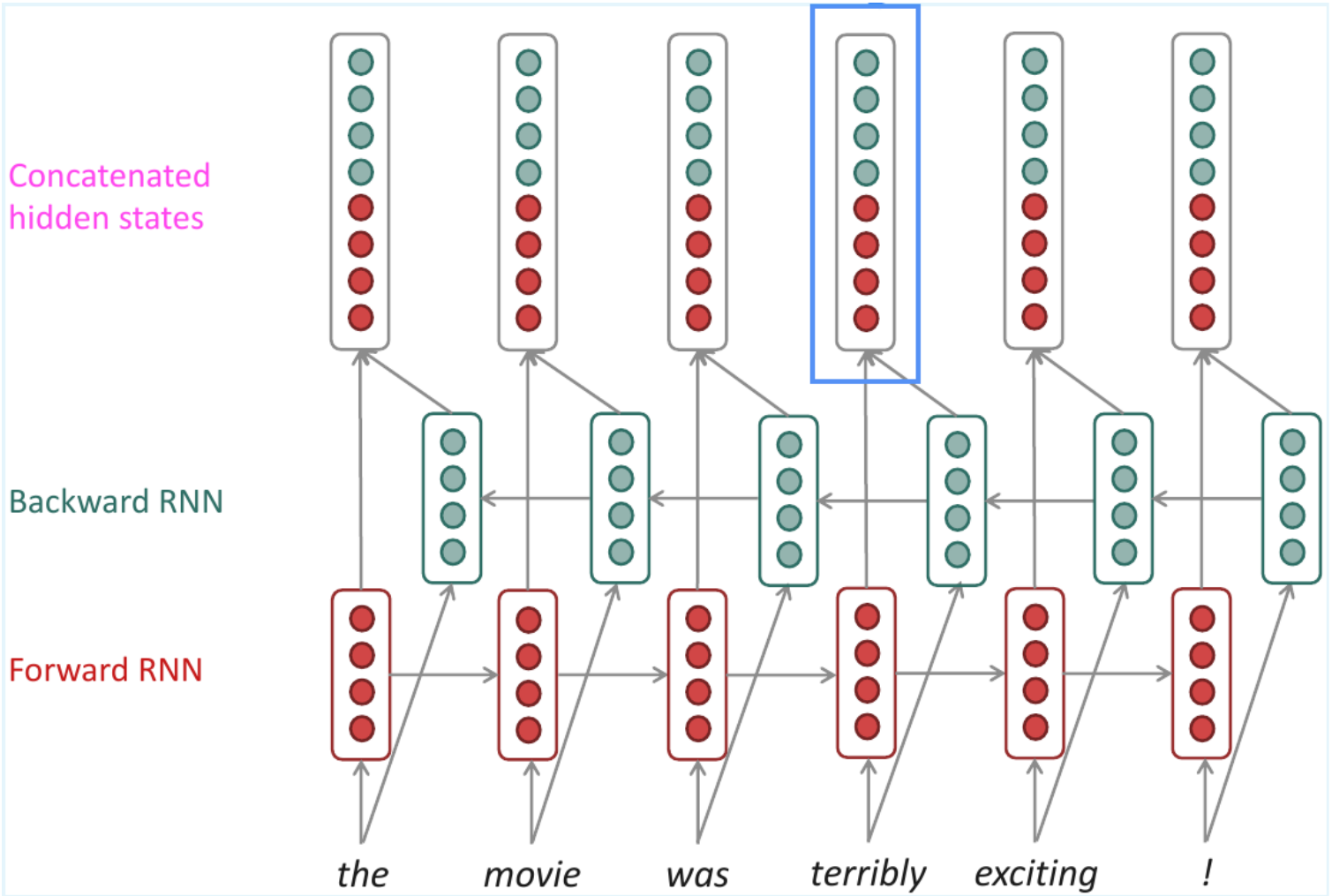
Section 5

NLP

Statistics & Data Science

Some token representations need evidence on both sides

A left-to-right RNN represents token t using only the prefix. When the whole input is available, useful evidence may also appear to the right.



A bidirectional representation concatenates two states

Assume that each direction uses hidden-state dimension m. Run one recurrence in each direction:

$$\overrightarrow{\mathbf{h}}_t = f_{\rightarrow}(\overrightarrow{\mathbf{h}}_{t-1}, \mathbf{x}_t),$$
$$\overleftarrow{\mathbf{h}}_t = f_{\leftarrow}(\overleftarrow{\mathbf{h}}_{t+1}, \mathbf{x}_t).$$

At position t, concatenate the two states:

$$\mathbf{h}_t^{\mathrm{bi}} = \left[\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t\right] \in \mathbb{R}^{2m}.$$

- $\overrightarrow{\mathbf{h}}_t$: summarizes the prefix $x_1, \ldots, x_t$
- $\overleftarrow{\mathbf{h}}_t$: summarizes the suffix $x_t, \ldots, x_T$

Bidirectional context is valid only when the full input is available

- Bidirectional RNNs require access to the **entire input sequence**.
  - They cannot define a causal next-token predictor if the backward state has already read the unknown future.

Bidirectional context is valid only when the full input is available

- Bidirectional RNNs require access to the **entire input sequence**.
  - They cannot define a causal next-token predictor if the backward state has already read the unknown future.
- When the full sequence is available, bidirectional states are useful for **encoding tasks** such as tagging or classification.

Bidirectional context is valid only when the full input is available

- Bidirectional RNNs require access to the **entire input sequence**.
  - They cannot define a causal next-token predictor if the backward state has already read the unknown future.
- When the full sequence is available, bidirectional states are useful for **encoding tasks** such as tagging or classification.
- Modern connection: BERT also learns representations from both left and right context, although it uses masked-token training and Transformer layers rather than an RNN.

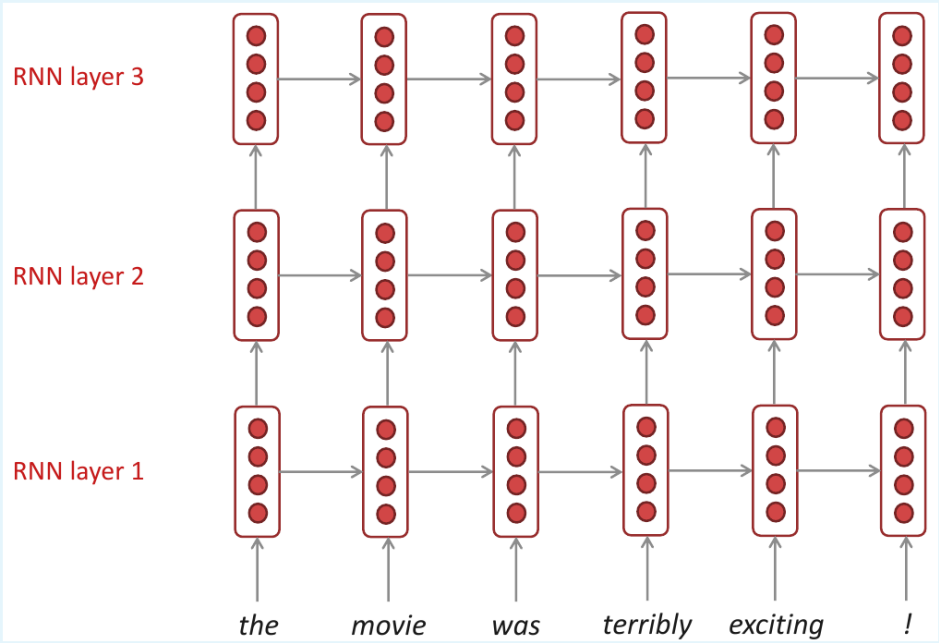
Stacked RNNs add depth to the state sequence

With one recurrent layer, the model produces one level of contextual features. Stacking lets higher layers transform those features again before prediction.

For layer $\ell = 1, \ldots, L$,

$$\mathbf{h}_t^{(\ell)} = f_\ell\left(\mathbf{h}_{t-1}^{(\ell)}, \mathbf{h}_t^{(\ell-1)}\right), \qquad \mathbf{h}_t^{(0)} = \mathbf{x}_t.$$

- Time recurrence moves horizontally within one layer.
- Stacking moves vertically from layer $\ell - 1$ to layer ℓ.



More recurrent layers increase capacity and optimization difficulty

- Each layer adds another learned transformation of the state sequence.
  - More layers can represent more complex functions than one layer.
  - Width and depth are different capacity choices.
- Additional layers also lengthen the optimization path.
  - Training becomes more expensive and can become less stable.
  - Residual or skip connections can make deeper stacks easier to optimize.

More depth increases capacity, but it also makes optimization harder.

LSTMs became widely used before Transformers

- **By the mid-2010s, LSTM-based systems achieved strong results in**
  - speech recognition and language modeling;
  - machine translation; and
  - image captioning and other sequence tasks.

- **Machine translation shows the later architectural shift**
  - WMT 2016: many leading neural systems were RNN-based
  - WMT 2019: Transformer-based systems had become overwhelmingly common

Sources:

- Bojar et al., 2016 (WMT16)
- Barrault et al., 2019 (WMT19)

Four models make four different context assumptions

Model	Context representation	Statistical sharing	Main limitation
n-gram	Last $n-1$ discrete tokens	Across identical contexts	Sparse fixed contexts
Fixed-window neural LM	Concatenated embeddings	Across similar words	Context length fixed
Vanilla RNN	Recurrent hidden state	Across words and positions	Long dependencies hard to train
LSTM	Gated recurrent memory	Across words and positions	Sequential computation remains

The progression changes how context is represented; the next-token probability remains the common language-model object.